

Zmienne

```
In [1]: # To jest komentarz
        """
        To jest komentarz.
        Może zawierać wiele linijek.
        Stosowany w dokumentacji klas i funkcji.
        """
```

```
# Wyświetlanie tekstu
print("Hello world")
```

Hello world

```
In [2]: # Tworzenie zmiennych
        # Liczba całkowita
        zmienna = 2

        # Liczba rzeczywista
        rzeczywista = 4.5
        rzeczywista = float(34) # konwersja danych
        rzeczywista = 34.0

        # zmienna tekstowa string
        zmiennaTekstowa1 = "jakis tekst"
        zmiennaTekstowa2 = "to jest nowa zmienna"

        # zmienna typu bool
        is_active = True

        # Łączenie i wyświetlanie tekstu
        print(zmiennaTekstowa1 + " " + zmiennaTekstowa2)
```

jakis tekst to jest nowa zmienna

```
In [3]: # Nie można bezpośrednio łączyć zmiennych tekstowych i liczbowych
        print(zmienna + zmiennaTekstowa1)
```

```
-----
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_6962/3229552851.py in <module>
      1 # Nie można bezpośrednio łączyć zmiennych tekstowych i liczbowych
----> 2 print(zmienna + zmiennaTekstowa1)
```

TypeError: unsupported operand type(s) for +: 'int' and 'str'

Formatowanie tekstu i obsługa napisów

```
In [4]: # Łączenie napisów
        tekst = "witaj" + " " + "swiecie"
```

```
In [5]: # Operacja "mnożenia" napisów pozwala na wygenerowanie powtarzających się
        # sekwencji danego napisu
        witaj = "witaj" * 10
        print(witaj)
```

witaj witaj witaj witaj witaj witaj witaj witaj witaj witaj

```
In [7]: print("Tablice o powtarzającym się ciągu elementów")
        print([4,5] * 4)
```

Tablice o powtarzającym się ciągu elementów
[4, 5, 4, 5, 4, 5, 4, 5]

```
In [8]: """
        Podstawowe specyfikatory do formatowania napisów
        %s - napis lub każdy inny obiekt mający reprezentację w formie napisu
        %d - liczby całkowite
        %f - liczby rzeczywiste
        %<ilosc cyfr> - liczba zmiennie przecinkowa ze stałą liczbą cyfr po kropce
        """
        lista = [3.54, 1.2353, 3.12]

        # Dla większej liczby wyświetlanych zmiennych, zmienne powinny znaleźć się po
        # znaku % w nawiasie
        print("Kolejne elementy: %f, %.1f, %d" % (lista[0], lista[1], lista[2]))
```

```

print("Element listy %f oraz typ %s" % (lista[0], type(lista[0])))
print("Element listy {:f} oraz jej typ {}".format(lista[0], type(lista[0])))
print()

# Dla pojedynczej zmiennej nie ma konieczności umieszczania jej w nawiasie
print("Element %f" % lista[0])
print("Element {:.f}".format(lista[0]))
print()

# Ograniczenie wyświetlania liczb znajdujących się po przecinku przy
# wyświetlaniu
print("Element %.1f" % lista[0])
print("Element {:.1f}".format(lista[0]))

Kolejne elementy: 3.540000, 1.2, 3
Element listy 3.540000 oraz typ <class 'float'>
Element listy 3.540000 oraz jej typ <class 'float'>

Element 3.540000
Element 3.540000

Element 3.5
Element 3.5

```

```

In [9]: # Sprawdzenie liczby przechowywanych znaków w zmiennej tekstowej. Podobnie jak
# dla tablic wykorzystywana jest
# funkcja len(nazwa_zmiennej)
tekst = "To jest przykładowy tekst."
print(len(tekst))

# zwrócenie pierwszego występującego indeksu danego znaku lub ciąg znaków
index = tekst.index("jest")
print("index: %d" % index)

26
index: 3

```

```

In [10]: # Zliczenie liczby występujących znaków lub ciąg znaków
print("Liczba występujących ciągów znaków 'k': %s" % tekst.count("k"))

Liczba występujących ciągów znaków 'k': 2

```

```

In [11]: # Dostęp do pojedynczego znaku
print(tekst[1])

o

```

```

In [12]: # Dostęp do fragmentu tekstu znajdującego się pomiędzy wskazanymi indeksami
print(tekst[2:5])

je

```

```

In [13]: # Wyświetlenie całego napisu od początku do wskazanego indeksu znaku
print(tekst[0:5])
print(tekst[:5])

To je
To je

```

```

In [14]: # Wyświetlenie całego napisu od wskazanego znaku do końca tekstu
print(tekst[5:len(tekst)])
print(tekst[5:])

st przykładowy tekst.
st przykładowy tekst.

```

```

In [15]: # Można również liczyć i wyświetlać znaki od końca. W tym celu podaje się
# ujemne wartości indeksów
print(tekst[-1]) # Wyświetlenie ostatniego znaku
print(tekst[-5:-3]) # Wyświetlenie znaków pomiędzy 5 (włącznie) pozycja
# od końca a 3 (włącznie)
print(tekst[-5:]) # Wyświetlenie znaków od 5 (włącznie) pozycji od końca
# do samego końca
print(tekst[:-5]) # Wyświetlenie wszystkich znaków od początek do 5 znaku
# od końca (pomijany jest 5 znak od końca)

.
ek
ekst.
To jest przykładowy t

```

```

In [16]: # Zamiana wszystkich liter na duże lub małe
print(tekst.upper())

```

```
print(tekst.lower())
```

TO JEST PRZYKŁADOWY TEKST.
to jest przykładowy tekst.

```
In [17]: # Sprawdzenie czy napis zaczyna sie lub konczy od podanego ciagu znakow
print(tekst.startswith("To "))
print(tekst.startswith("tekst."))
print(tekst.endswith("To "))
print(tekst.endswith("tekst."))
```

True
False
False
True

```
In [18]: # Rozdzielanie napisu i zapisywanie go w tablicy np. w poniższym przykładzie
# spacja odpowiada za rozdzielenie tekstu
tablica_rozdzielona = tekst.split(" ")
print(tablica_rozdzielona)
```

['To', 'jest', 'przykładowy', 'tekst.']

List

```
In [19]: """
list - jest to kolekcja wielu elementów.
Elementy listy są:
- posortowane,
- edytowalne (dodawanie, usuwanie, zmienianie wartości),
- indeksowane, pierwszy element ma index 0.
Na liście można przechowywać zduplikowane wartości.
Zapisywane są w kwadratowych nawiasach.
"""

# inicjalizacja listy wraz z wartościami
lista_punktow = ["A", "C", "D"]

# mogą przechowywać elementy różnego typu
lista_danych = ["A", "C", 3, "D", 3, True]
```

```
In [20]: # Tworzenie list
lista = []
# dodawanie nowych wartości, użycie metody append
lista.append(23.2)
lista.append(12.3)
lista.append(4)
```

```
In [21]: # Sprawdzenie długości
print("Długość listy")
print(len(lista))

# Wyświetlanie wartości dla podanego indeksu
print("Kolejne wartosci:")
print(lista[0]) # A
print(lista[1]) # C
print(lista[2]) # D
print("---")
print(lista_danych[-1]) # True
print(lista_danych[-3]) # "D"
```

Długość listy
3
Kolejne wartosci:
23.2
12.3
4

True
D

```
In [22]: # Inny sposob na wyswietlenie kolejnych elementow listy poprzez
# konwersje elementow na string
print("Kolejne elementy: " + str(lista[0]) + ", " + str(lista[1])
      + ", " + str(lista[2]))

# Wyświetlenie elementu i jego typu
print("Liczba: " + str(lista[0]) + " typ " + str(type(lista[0])))
print("Liczba: " + str(lista[1]) + " typ " + str(type(lista[1])))
print("Liczba: " + str(lista[2]) + " typ " + str(type(lista[2])))
```

Kolejne elementy: 23.2, 12.3, 4
Liczba: 23.2 typ <class 'float'>
Liczba: 12.3 typ <class 'float'>

```
In [23]: # Lista ograniczona indeksem początkowym i końcowym
lista_danych = ["A", "C", 3, "D", 3, True]
print(lista_danych[2:4]) # element o indeksie 2 znajduje się na liście,
                        # natomiast o indeksie 4 jest końcowym i nie pojawia się na niej

[3, 'D']
```

```
In [24]: # zwrócenie wszystkich elementów listy do podanego indeksu
lista_danych = ["A", "C", 3, "D", 3, True]
print(lista_danych[:4])

['A', 'C', 3, 'D']
```

```
In [25]: # zwrócenie wszystkich elementów listy od podanego indeksu
lista_danych = ["A", "C", 3, "D", 3, True]
print(lista_danych[2:])

[3, 'D', 3, True]
```

```
In [26]: # Rozszerzenie listy o inną listę
lista_a = [1,2,3]
lista_b = ["a", "b"]
print("przed rozszerzeniem \n lista_a=" + str(lista_a) + "\n")

lista_a.extend(lista_b)
print("po rozszerzeniu \n lista_a=" + str(lista_a) + "\n")

print("po modyfikacji elementu z lista_b")
lista_b[0] = "c"
print(" lista_a=" + str(lista_a) + " lista_b=" + str(lista_b) + "\n")

przed rozszerzeniem
lista_a=[1, 2, 3]

po rozszerzeniu
lista_a=[1, 2, 3, 'a', 'b']

po modyfikacji elementu z lista_b
lista_a=[1, 2, 3, 'a', 'b'] lista_b['c', 'b']
```

```
In [27]: # usuwanie elementu po wartości
lista_danych = ["A", "C", 3, "D", 3, True]
lista_danych.remove("D")
print(lista_danych)

['A', 'C', 3, 3, True]
```

```
In [29]: # usuwanie elementu z listy po indeksie
lista_danych = ["A", "C", 3, "D", 3, True]
lista_danych.pop(1)
print(lista_danych)

['A', 3, 'D', 3, True]
```

```
In [ ]: """
Niektóre metody:
- clear() - wyczyszczenie listy
- copy() - zwraca kopie aktualnej listy
- count(wartosc) - zwraca liczbę występujących elementów o podanej
wartości
- index(wartosc) - zwraca index pierwszego elementu o podanej
wartości
- sort() - sortowanie listy, domyślny argument sortujący rosnąco
reverse=False, dla sortowania malejącego należy podać
reverse=True

"""
```

```
In [31]: # Edycja listy, która nie jest kopią innej listy. Przypisanie referencji do lista_a do zmiennej lista_b.
lista_a = [2,4,6,5,3]
lista_b = lista_a
print("przed modyfikacją \n lista_a=" + str(lista_a) + " lista_b=" + str(lista_b))
lista_b[0] = -1
print("po modyfikacji elementu o indeksie 0 dla lista_b \n lista_a=" + str(lista_a) + " lista_b=" + str(lista_b))
print("Element o indeksie 0 dla obu list wynosi -1")
```

```

przed modyfikacją
lista_a=[2, 4, 6, 5, 3] lista_b=[2, 4, 6, 5, 3]
po modyfikacji elementu o indeksie 0 dla lista_b
lista_a=[2, 4, 6, 5, 3] lista_b=[-1, 4, 6, 5, 3]

```

```

In [32]: # Edycja listy, która jest kopią innej listy. Przypisanie kopii lista_a do zmiennej lista_b.
lista_a = [2,4,6,5,3]
lista_b = lista_a.copy()
print("przed modyfikacją \n lista_a=" + str(lista_a) + " lista_b=" + str(lista_b))
lista_b[0] = -1
print("po modyfikacji elementu o indeksie 0 dla lista_b \n lista_a=" + str(lista_a) + " lista_b=" + str(lista_b))

```

```

przed modyfikacją
lista_a=[2, 4, 6, 5, 3] lista_b=[2, 4, 6, 5, 3]
po modyfikacji elementu o indeksie 0 dla lista_b
lista_a=[2, 4, 6, 5, 3] lista_b=[-1, 4, 6, 5, 3]

```

```

In [33]: # Czyszczenie listy, która nie jest kopią innej listy
lista_a = [2,4,6,5,3]
lista_b = lista_a
print("przed czyszczeniem \n lista_a=" + str(lista_a) + " lista_b=" + str(lista_b))
lista_b.clear()
print("po wyczyszczeniu \n lista_a=" + str(lista_a) + " lista_b=" + str(lista_b))

```

```

przed czyszczeniem
lista_a=[2, 4, 6, 5, 3] lista_b=[2, 4, 6, 5, 3]
po wyczyszczeniu
lista_a=[] lista_b=[]

```

```

In [34]: # Czyszczenie listy, która jest kopią innej listy
lista_a = [2,4,6,5,3]
lista_b = lista_a.copy()
print("przed czyszczeniem \n lista_a=" + str(lista_a) + " lista_b=" + str(lista_b))
lista_b.clear()
print("po wyczyszczeniu \n lista_a=" + str(lista_a) + " lista_b=" + str(lista_b))

```

```

przed czyszczeniem
lista_a=[2, 4, 6, 5, 3] lista_b=[2, 4, 6, 5, 3]
po wyczyszczeniu
lista_a=[2, 4, 6, 5, 3] lista_b=[]

```

```

In [35]: lista_a = [2,4,6,5,3]
lista_b = lista_a.copy()
print("przed czyszczeniem \n lista_a=" + str(lista_a) + " lista_b=" + str(lista_b))
lista_a.clear()
print("po wyczyszczeniu \n lista_a=" + str(lista_a) + " lista_b=" + str(lista_b))

```

```

przed czyszczeniem
lista_a=[2, 4, 6, 5, 3] lista_b=[2, 4, 6, 5, 3]
po wyczyszczeniu
lista_a=[] lista_b=[2, 4, 6, 5, 3]

```

```

In [36]: # Zliczanie liczby podanych elementów występujących w liście.
lista_a = [2,4,5,7, "L",2,4,7,4,2, "L", "j", True]
print("liczba wystąpień wartości 2: " + str(lista_a.count(2)))
print("liczba wystąpień znaku L: " + str(lista_a.count("L")))
print("liczba wystąpień znaku Z: " + str(lista_a.count("Z")))

```

```

liczba wystąpień wartości 2: 3
liczba wystąpień znaku L: 2
liczba wystąpień znaku Z: 0

```

```

In [37]: # Zwracanie indeksu pierwszego występującego elementu na liście
lista_a = [2,4,5,7, "L",2,4,7,4,2, "L", "j"]
print("Indeks pierwszego występującego elementu 2: " + str(lista_a.index(2)))
print("Indeks pierwszego występującego elementu L: " + str(lista_a.index("L")))

```

```

Indeks pierwszego występującego elementu 2: 0
Indeks pierwszego występującego elementu L: 4

```

```

In [38]: print("Indeks dla elementu, którego nie ma na liście np. Z: " + str(lista_a.index("Z")))

```

```

-----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_6962/1824898570.py in <module>
----> 1 print("Indeks dla elementu, którego nie ma na liście np. Z: " + str(lista_a.index("Z")))

ValueError: 'Z' is not in list

```

```

In [39]: # sortowanie listy - liczby
lista_a = [2,4,5,7,2,4,7,4,2]
print("Sortowanie rosnące")

```

```
lista_a.sort()
print(lista_a)

print("Sortowanie malejące")
lista_a.sort(reverse=True)
print(lista_a)
```

```
Sortowanie rosnące
[2, 2, 2, 4, 4, 4, 5, 7, 7]
Sortowanie malejące
[7, 7, 5, 4, 4, 4, 2, 2, 2]
```

```
In [40]: # sortowanie listy - tekst
lista_a = ["abc", "cvf", "def", "qwerty", "aav"]
print("Sortowanie rosnące")
lista_a.sort()
print(lista_a)

print("Sortowanie malejące")
lista_a.sort(reverse=True)
print(lista_a)
```

```
Sortowanie rosnące
['aav', 'abc', 'cvf', 'def', 'qwerty']
Sortowanie malejące
['qwerty', 'def', 'cvf', 'abc', 'aav']
```

```
In [41]: # niemożliwe sortowanie listy - różne typy
lista_a = [2,4,5,7, "L",2,4,7,4,2, "L", "j", True]
lista_a.sort()
```

```
-----
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_6962/483531035.py in <module>
      1 # niemożliwe sortowanie listy - różne typy
      2 lista_a = [2,4,5,7, "L",2,4,7,4,2, "L", "j", True]
----> 3 lista_a.sort()
```

```
TypeError: '<' not supported between instances of 'str' and 'int'
```

Tuple

```
In [42]: """
tuple - jest to kolekcja wielu elementów.
Elementy są:
- posortowane,
- nieedytowalne,
- indeksowane.
Tuple pozwala na przechowywanie zduplikowanych wartości.
Zapisywane są w okrągłych nawiasach.
"""
punkty = ("A", "C", "D", "C")

# mogą przechowywać elementy różnego typu
dane = ("A", "C", 3, "D", 3, True)
```

```
In [43]: # Zwraca liczbę wystąpień danego elementu
dane.count("C")
```

```
Out[43]: 1
```

```
In [44]: # Zwraca index pierwszego znalezionej elementu pasującego do
# przekazanej wartości
dane.index("C")
```

```
Out[44]: 1
```

```
In [45]: # Zwracanie elementu o podanym indeksie
dane = ("A", "C", 3, "D", 3, True)
print(dane[3])
```

```
D
```

```
In [46]: # Edycja elementu - błąd
dane = ("A", "C", 3, "D", 3, True)
dane[0] = "zmiana"
```

```

-----
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_6962/2709152694.py in <module>
      1 # Edycja elementu - błąd
      2 dane = ("A", "C", 3, "D", 3, True)
----> 3 dane[0] = "zmiana"

```

Set

```

In [47]: """
set - jest to kolekcja wielu elementów.
Elementy są:
- nieposortowane,
- nieedytowalne (ale można dodawać i usuwać elementy),
- nieindexowane.
Zbiór nie zawiera zduplikowanych wartości.
Zapisywane są w sześciennych nawiasach.
"""
punkty = {"A", "C", "D"}

# mogą przechowywać elementy różnego typu
dane = {"A", "C", 3, "D", True}

```

```

In [48]: # dodanie nowego elementu
dane.add(34)

```

```

In [ ]: """
Niektóre metody:
- clear() - wyczyszczenie listy
- copy() - zwraca kopie aktualnej listy
- pop() - usuwa losowy element ze zbioru
- remove(element) - usuwa przekazany element jako argument ze zbioru
- difference(set2) - zwraca różnicę elementów pomiędzy pierwszym
                    a drugim zbiorem set2
- update(set2) - aktualizuje zbiór poprzez dodanie wartości z
                 drugiego zbioru set2
"""

```

```

In [49]: # Aktualizacja zbioru - update
zbior_1 = {"a", "b", "c", "d", "e"}
zbior_2 = {"b", "e", "g", "h"}
zbior_1.update(zbior_2)
print(zbior_1)
print("modyfikacja zbioru 2")
zbior_2.remove("e")
print(zbior_1)
print("Modyfikacja zbior_2 po aktualizacji zbior_1 nie wpływa na zbior_1")

{'b', 'g', 'd', 'a', 'c', 'h', 'e'}
modyfikacja zbioru 2
{'b', 'g', 'd', 'a', 'c', 'h', 'e'}
Modyfikacja zbior_2 po aktualizacji zbior_1 nie wpływa na zbior_1

```

```

In [50]: # Różnica zbiorów - difference
zbior_1 = {"a", "b", "c", "d", "e"}
zbior_2 = {"b", "e", "g", "h"}
print("zbior_1 - zbior_2: " + str(zbior_1.difference(zbior_2)))
print("zbior_2 - zbior_1: " + str(zbior_2.difference(zbior_1)))

zbior_1 - zbior_2: {'d', 'a', 'c'}
zbior_2 - zbior_1: {'g', 'h'}

```

```

In [51]: # Zatrzymanie elementów w zbiorze, które występują w innym zbiorze, usunięcie pozostałych
# Modyfikacja zbior_1
zbior_1 = {"a", "b", "c", "d", "e"}
zbior_2 = {"b", "e", "g", "h"}
zbior_1.intersection_update(zbior_2)
print(zbior_1)

{'e', 'b'}

```

```

In [52]: # Utworzenie nowego zbioru z elementami wspólnymi obu zbiorów
zbior_1 = {"a", "b", "c", "d", "e"}
zbior_2 = {"b", "e", "g", "h"}
nowy_zbior = zbior_1.intersection(zbior_2)
print("zbior_1=" + str(zbior_1))
print("zbior_2=" + str(zbior_2))
print("nowy_zbior=" + str(nowy_zbior))

```

```
zbior_1={'b', 'd', 'a', 'c', 'e'}
```

```
In [53]: # Utworzenie nowego zbioru z elementami wspólnymi obu zbiorów
# Wartości True i 1 są sobie równoważne.
# W nowym zbiorze pojawia się 1 jako część wspólna.
zbior_1 = {"a", "b", "c", "d", "e", True}
zbior_2 = {"b", "e", "g", "h", 1}
nowy_zbior = zbior_1.intersection(zbior_2)
print(nowy_zbior)
```

```
{1, 'e', 'b'}
```

```
In [54]: # Zatrzymanie elementów różniących oba zbiory.
# Elementy wspólne są pomijane.
# Aktualizacja zbior_1
zbior_1 = {"a", "b", "c", "d", "e"}
zbior_2 = {"b", "e", "g", "h"}
zbior_1.symmetric_difference_update(zbior_2)
print(zbior_1)
```

```
{'g', 'd', 'a', 'c', 'h'}
```

```
In [55]: # Utworzenie nowego zbioru z elementów, które nie powtarzają się w obu zbiorach.
zbior_1 = {"a", "b", "c", "d", "e"}
zbior_2 = {"b", "e", "g", "h"}
nowy_zbior = zbior_1.symmetric_difference(zbior_2)
print("zbior_1=" + str(zbior_1))
print("zbior_2=" + str(zbior_2))
print("nowy_zbior=" + str(nowy_zbior))
```

```
zbior_1={'b', 'd', 'a', 'c', 'e'}
zbior_2={'g', 'h', 'e', 'b'}
nowy_zbior={'g', 'd', 'a', 'c', 'h'}
```

```
In [56]: # Usuwanie elementu ze zbioru - remove
zbior = {"a", "b", "c", "d", "e"}
zbior.remove("c")
print(zbior)
```

```
{'b', 'd', 'a', 'e'}
```

```
In [57]: # Usuwanie elementu ze zbioru - remove - błąd - element nie istnieje
zbior = {"a", "b", "c", "d", "e"}
zbior.remove("z")
```

```
-----
KeyError                                Traceback (most recent call last)
/tmp/ipykernel_6962/706358594.py in <module>
      1 # Usuwanie elementu ze zbioru - remove - błąd - element nie istnieje
      2 zbior = {"a", "b", "c", "d", "e"}
----> 3 zbior.remove("z")

KeyError: 'z'
```

```
In [58]: # Usuwanie elementu ze zbioru - discard
zbior = {"a", "b", "c", "d", "e"}
zbior.discard("c")
print(zbior)
```

```
{'b', 'd', 'a', 'e'}
```

```
In [59]: # Usuwanie elementu ze zbioru - discard - brak błędu - element nie istnieje
zbior = {"a", "b", "c", "d", "e"}
zbior.discard("z")
print(zbior)
```

```
{'b', 'd', 'a', 'c', 'e'}
```

Dict

```
In [60]: """
dict - jest to kolekcja wielu elementów.
Elementy są:
- nieposortowane (od wersji 3.7 wprowadzono sortowanie),
- edytowalne,
- zawierają unikalne klucze.

Jest to słownik przechowujący elementy pod postacią klucz wartość.
Zapisany w nawiasach sześciennych.
```



```
"""
punkty = {
    "A": (1,3),
    "B": (2,7),
    "C": (6,3),
    "nazwa": "zbior"
}

# dodanie nowego elementu do słownika
punkty["nowy_klucz"] = 31
```

```
In [61]: # Pobranie elementu ze słownika
print(punkty["A"])
```

```
(1, 3)
```

```
In [ ]: """
Niektóre metody:
- clear() - wyczyszczenie listy
- copy() - zwraca kopie aktualnej listy
- pop(klucz) - usuwa element o podanym kluczu
- keys() - zwraca listę z kluczami elementów
- values() - zwraca listę z wartościami elementów słownika
- update(dict2) - aktualizuje słownik o podane wartości. Jeśli niewystępowały to
    są dodane nowe, a jeśli już były dodane klucze to wartości dla
    nich są aktualizowane.
"""
```

```
In [62]: # lista dostępnych kluczy w słowniku
punkty.keys()
```

```
Out[62]: dict_keys(['A', 'B', 'C', 'nazwa', 'nowy_klucz'])
```

```
In [63]: # lista dostępnych wartości w słowniku
punkty.values()
```

```
Out[63]: dict_values([(1, 3), (2, 7), (6, 3), 'zbior', 31])
```

```
In [64]: # Lista elementów słownika w postaci (klucz, wartość)
punkty.items()
```

```
Out[64]: dict_items([('A', (1, 3)), ('B', (2, 7)), ('C', (6, 3)), ('nazwa', 'zbior'), ('nowy_klucz', 31)])
```

```
In [65]: # Usuwanie elementu słownika o podanym kluczu
```

```
punkty = {
    "A": (1,3),
    "B": (2,7),
    "C": (6,3),
}

print("przed " + str(punkty))
punkty.pop("B")
print("po " + str(punkty))
```

```
przed {'A': (1, 3), 'B': (2, 7), 'C': (6, 3)}
po {'A': (1, 3), 'C': (6, 3)}
```

```
In [66]: # Edycja elementu słownika
```

```
punkty = {
    "A": (1,3),
    "B": (2,7),
    "C": (6,3),
}
punkty["B"] = (1,1)
print(punkty)
```

```
{'A': (1, 3), 'B': (1, 1), 'C': (6, 3)}
```

```
In [67]: # Przypisanie referencji do słownika do innej zmiennej
# słownik punkty - jest modyfikowany
```

```
punkty = {
    "A": (1,3),
    "B": (2,7),
    "C": (6,3),
}

nowe_punkty = punkty

print("Przed modyfikacją \n " + "punkty=" + str(punkty) + " nowe_punkty=" + str(nowe_punkty))

nowe_punkty["C"] = (-1,3)
nowe_punkty["D"] = (0,0)
```

```
print("Przed modyfikacją \n " + "punkty=" + str(punkty) + " nowe_punkty=" + str(nowe_punkty))
Przed modyfikacją
punkty={'A': (1, 3), 'B': (2, 7), 'C': (6, 3)} nowe_punkty={'A': (1, 3), 'B': (2, 7), 'C': (6, 3)}
Przed modyfikacją
punkty={'A': (1, 3), 'B': (2, 7), 'C': (-1, 3), 'D': (0, 0)} nowe_punkty={'A': (1, 3), 'B': (2, 7), 'C': (-1, 3), 'D': (0, 0)}
```

```
In [68]: # Przypisanie kopii słownika do innej zmiennej
# słownik punkty - nie jest modyfikowany
punkty = {
    "A": (1,3),
    "B": (2,7),
    "C": (6,3),
}

nowe_punkty = punkty.copy()

print("Przed modyfikacją \n " + "punkty=" + str(punkty) + " nowe_punkty=" + str(nowe_punkty))

nowe_punkty["C"] = (-1,3)
nowe_punkty["D"] = (0,0)
print("Przed modyfikacją \n " + "punkty=" + str(punkty) + " nowe_punkty=" + str(nowe_punkty))

Przed modyfikacją
punkty={'A': (1, 3), 'B': (2, 7), 'C': (6, 3)} nowe_punkty={'A': (1, 3), 'B': (2, 7), 'C': (6, 3)}
Przed modyfikacją
punkty={'A': (1, 3), 'B': (2, 7), 'C': (6, 3)} nowe_punkty={'A': (1, 3), 'B': (2, 7), 'C': (-1, 3), 'D': (0, 0)}
```

```
In [69]: # Zagnieżdżenie słowników
paths = {
    1: {
        "A": (1,1),
        "B": (2,3)
    },
    2: {
        "A": (1,1),
        "B": (2,3),
        "C": (-1,9),
        "D": (0,0)
    },
    3: {
        "A": (1,1),
        "C": (-1,9)
    }
}

# Edycja punktu D
paths[2]["D"] = (-3, -3)
print(paths)

{1: {'A': (1, 1), 'B': (2, 3)}, 2: {'A': (1, 1), 'B': (2, 3), 'C': (-1, 9), 'D': (-3, -3)}, 3: {'A': (1, 1), 'C': (-1, 9)}}
```

Sprawdzenie typu zmiennej

```
In [70]: # Sprawdzenie typu zmiennej
print(type(zmienna))
print(type(rzeczywista))
print(type(zmiennaTekstowa1))

<class 'int'>
<class 'float'>
<class 'str'>
```

Operacje na obiektach

```
In [71]: """
Operacje matematyczne.
- dodawania(+),
- odejmowania(-),
- mnożenia(*),
- dzielenia(/),
- operacji modulo(%),
- potęgowania (**),
- dzielenia bez reszty (/).

"""

# Podnoszenie do potęgi
```

```

kwadrat = 3**2
szescian = 6**3
print("kwadrat " + str(kwadrat))
print("sześcián " + str(szescian))

# dzielenie bez reszty
wynik = 15//2
print("wynik dzielenia 15/2 bez reszty: " + str(wynik))

kwadrat 9
sześcián 216
wynik dzielenia 15/2 bez reszty: 7

```

```

In [72]: """
Operatory porównania.
- równe (==),
- różne (!=),
- większe niż (>),
- mniejsze niż (<),
- większe lub równe (>=),
- mniejsze lub równe (<=),
"""
a = 4
b = 7
print("a == 4 " + str(a == 4)) # True
print("a != 4 " + str(a != 4)) # False
print("a == 5 " + str(a == 5)) # False
print("a > 1 " + str(a > 1)) # True
#zmiana wartości wyrażenia na przeciwną
print("not a == 4 " + str(not a == 4))

a == 4 True
a != 4 False
a == 5 False
a > 1 True
not a == 4 False

```

```

In [73]: """
Operatory logiczne.
- and - iloczyn logiczny
- or - suma logiczna
- not - zaprzeczenie
"""
print("a == 4 and b > 3 " + str(a == 4 and b > 3)) #True
print("a == 2 or b > 2 " + str(a == 2 or b > 2)) #True

a == 4 and b > 3 True
a == 2 or b > 2 True

```

```

In [74]: # Do sprawdzenia czy podany operator znajduje się w zbiorze służy operator in
tablica = ["abc", "def", "ghi"]
print("def" in tablica) # true
print("test" in tablica) # false

True
False

```

```

In [75]: # Do sprawdzenia czy zmienne wskazują na ten sam obszar w pamięci służy operator is
x = [5, 3, 3]
y = [2, 6, 3]

tablica2 = [4, x, y]

print("x == tablica2[1] " + str(x == tablica2[1])) # True
print("x is tablica2[1] " + str(x is tablica2[1])) # True
print("x is tablica2 " + str(x is tablica2)) # False

x == tablica2[1] True
x is tablica2[1] True
x is tablica2 False

```

Instrukcja warunkowa If ... Else

```

In [ ]: # Dla instrukcji warunkowych należy pamiętać o wcięciach w programie
if warunek: # warunek z wykorzystaniem operatorów porównania i logicznych
    jakas_akcja
elif warunek2:
    jakas_inna_akcja
else: # wykonanie programu, gdy żaden z powyższych warunków nie został
    # spełniony.
    jakas_jeszcze_inna_akcja

```

```
In [76]: # Przykład użycia
a = 3
if a == 5:
    print("zmienna a jest rowna 5")
elif a > 5:
    print("zmienna a jest wieksza od 5")
else:
    print("zmienna a jest mniejsza od 5")
```

zmienna a jest mniejsza od 5

```
In [77]: # Zapis warunku w jednej linii
a = 15
b = 10
print("A") if a > b else print("B")
```

A

```
In [78]: # Sprawdzenie czy element znajduje się na liście
lista_a = [2,4,5,7, "L", 2,4,7,4,2, "L", "j", True]
if 2 in lista_a:
    print("2 znajduje się na liście")
```

2 znajduje się na liście

```
In [79]: lista_a = [2,4,5,7, "L", 2,4,7,4,2, "L", "j", True]
if lista_a:
    print("Lista nie jest pusta")
```

Lista nie jest pusta

```
In [80]: if []:
    print("Lista nie jest pusta")
else:
    print("Lista jest pusta")
```

Lista jest pusta

```
In [81]: # Sprawdzenie czy podany klucz znajduje się w słowniku
punkty = {
    "A": (1,3),
    "B": (2,7),
    "C": (6,3),
}

print("A" in punkty) # True
print("Z" in punkty) # False
```

True
False

```
In [82]: # Sprawdzenie czy słownik jest pusty

punkty = {
    "A": (1,3),
    "B": (2,7),
    "C": (6,3),
}

if punkty:
    print("Słownik nie jest pusty.")
```

Słownik nie jest pusty.

Pętle

For

```
In [83]: # for - pętla wykonuje się przez określoną liczbę iteracji. Nie wymaga
# ustawiania zmiennej indeksującej przy
# inicjalizacji.
liczby = [4, 2, 6, 4, 6]

for liczba in liczby:
    # liczba - zmienna w której przechowywany jest każdy kolejny element
    # tablicy "liczby". Pętla wykonuje się dopóki nie zostaną wyświetlone
    # wszystkie elementy z tablicy
    print(liczba)

4
2
6
4
6
```

```
In [84]: # iterowanie po kolejnych znakach tekstu
tekst = "To jest tekst"
for znak in tekst:
    print(znak)
```

T
o

j
e
s
t

t
e
k
s
t

```
In [85]: # iterowanie po kolejnych elementach listy
lista_danych = ["A", "C", 3, "D", 3, True]
for element_listy in lista_danych:
    print(element_listy)
```

A
C
3
D
3
True

```
In [86]: # iterowanie po kolejnych elementach listy z numeracją kolejnych elementów
lista_danych = ["A", "C", 3, "D", 3, True]
for index, element_listy in enumerate(lista_danych):
    print("index " + str(index) + " element_listy=" + str(element_listy))
```

index 0 element_listy=A
index 1 element_listy=C
index 2 element_listy=3
index 3 element_listy=D
index 4 element_listy=3
index 5 element_listy=True

```
In [87]: # Wykorzystanie pętli do tworzenia listy (list comprehension)
nowa_lista = [2*element_do_listy for element_do_listy in range(5)]
print(nowa_lista)
```

[0, 2, 4, 6, 8]

```
In [88]: # Iterowanie po kluczach słownika
punkty = {
    "A": (1,3),
    "B": (2,7),
    "C": (6,3),
}
for nazwa_punktu in punkty.keys():
    print(nazwa_punktu)
```

A
B
C

```
In [89]: # Iterowanie po wartościach słownika
punkty = {
    "A": (1,3),
```

```

    "B": (2,7),
    "C": (6,3),
}
for wspolrzedne_punktu in punkty.values():
    print(wspolrzedne_punktu)

(1, 3)
(2, 7)
(6, 3)

```

```

In [90]: # Iterowanie po kluczach i wartościach słownika
punkty = {
    "A": (1,3),
    "B": (2,7),
    "C": (6,3),
}
for nazwa_punktu, wspolrzedne_punktu in punkty.items():
    print("Punkt " + str(nazwa_punktu) + " ma wspolrzedne: " + str(wspolrzedne_punktu) + ".")

Punkt A ma wspolrzedne: (1, 3).
Punkt B ma wspolrzedne: (2, 7).
Punkt C ma wspolrzedne: (6, 3).

```

```

In [91]: # Liczby z wykorzystaniem instrukcji xrange mogą również zawierać się
# w określonym przedziale
for i in range(5):
    print(i)

0
1
2
3
4

```

```

In [92]: for i in range(5):
    pass # nie wpływa na działanie programu. Pass stosowane jest, gdy
    # z różnych powodów pętla, instrukcja warunkowa, funkcja nie posiada
    # zawartości. W przeciwnym wypadku pojawia się błąd.

```

```

In [93]: # brak pass powoduje błąd przy uruchomieniu programu
for i in range(5):
    File "/tmp/ipykernel_6962/350696600.py", line 2
        for i in range(5):
            ^
SyntaxError: unexpected EOF while parsing

```

```

In [94]: # instrukcja continue odpowiada za opuszczenie instrukcji znajdujących
# się poniżej i wraca do wykonywania kodu znajdującego się od początku
# pętli. Poniższa instrukcja wypisze tylko liczby parzyste
for x in range(30):
    # Sprawdź, czy x jest nieparzyste
    if x % 2 != 0:
        continue
    print(x)

0
2
4
6
8
10
12
14
16
18
20
22
24
26
28

```

```

In [95]: # iteracja w podanym zakresie o danym kroku
start = 2
stop = 30
krok = 3
for x in range(start, stop, krok):
    print(x)

```

```
2
5
8
11
14
17
20
```

```
In [96]: # Zagnieżdżone pętle
for x in [1,2,3]:
    for y in [4,5,6]:
        print(x,y)
```

```
1 4
1 5
1 6
2 4
2 5
2 6
3 4
3 5
3 6
```

While

```
In [97]: # while - pętla wykonuje się dopóki pewien warunek jest spełniony
i = 0
while i < 5:
    print(i)
    i += 1
```

```
0
1
2
3
4
```

```
In [98]: # instrukcja break używana jest do zakończenia działania pętli for
# i while
i = 0
while True: # pętla nieskończona
    print(i) # powoduje, że kolejne elementy wyświetlane są dalej
              # w linii, a nie nowym wierszu
    i += 1
    if i >= 3:
        break # gdy x >= 3 pętla zostaje przzerwana

print(i)
```

```
0
1
2
3
```

Funkcje

```
In [ ]: # Składnia
def nazwa_funkcji():
    # kod programu
    pass

def nazwa_funkcji(argument1, argument2):
    # można podać więcej argumentów po przecinku
    # kod programu
    pass

def nazwa_funkcji():
    # kod programu
    wartosc = 5
    return wartosc # funkcja zwraca jakąś wartość zapisaną w kodzie programu
```

```
In [99]: # Przykładowa funkcja wyświetlająca informację czy dana liczba jest
# podzielna przez 3
def podzielna_przez_3(liczba):
    if (liczba % 3) == 0:
        print("Podana liczba: {:d}, jest podzielna przez 3".format(liczba))
    else:
        print("Podana liczba: {:d}, nie jest podzielna przez 3".format(liczba))
```

```
In [100]: # Wywołanie funkcji
```

```
podzielna_przez_3(6)
podzielna_przez_3(2)
```

Podana liczba: 6, jest podzielna przez 3
Podana liczba: 2, nie jest podzielna przez 3

```
In [101]: def podzielna_przez_3_default_arg(liczba=5): # inicjalizacja argumentu domyślną
# wartością
if (liczba % 3) == 0:
    print("Podana liczba: {:d}, jest podzielna przez 3".format(liczba))
else:
    print("Podana liczba: {:d}, nie jest podzielna przez 3".format(liczba))
```

```
In [102]: # Wywołanie funkcji
# do funkcji z domyślnymi wartościami argumentów nie trzeba przekazywać
# ich wartości:
podzielna_przez_3_default_arg()
# ale można:
podzielna_przez_3_default_arg(2)
```

Podana liczba: 5, nie jest podzielna przez 3
Podana liczba: 2, nie jest podzielna przez 3

Importowanie modułów

```
In [103]: # importowanie biblioteki
import math
```

```
In [104]: # Wyświetlenie wszystkich dostępnych funkcji dla modułu math
dir(math)
```

Out[104]:


```
['_doc_',  
['_loader_',  
['_name_',  
['_package_',  
['_spec_',  
'acos',  
'acosh',  
'asin',  
'asinh',
```

```
In [105]: # importowanie konkretnych funkcji, klas z biblioteki  
from math import pi  
print(pi)
```

3.141592653589793

```
In [106]: # Wyświetlanie pomocy do funkcji z biblioteki  
help(math.log)
```

Help on built-in function log in module math:

```
log(...)  
log(x, [base=math.e])  
Return the logarithm of x to the given base.
```

If the base not specified, returns the natural logarithm (base e) of x.

Zmienne globalne

Znajdują się na zewnątrz funkcji lub klasy. Do ich modyfikacji w funkcji lub klasie wykorzystuje się słowo kluczowe **global**.

```
In [107]: x = 10  
  
def funkcja():  
    global x # wskazuje, że w funkcji będzie modyfikowana zmienna globalna x  
    x = 20  
  
funkcja()  
  
print(x)
```

20

```
In [108]: x = 10  
  
def funkcja():  
    # zmienna x widoczna tylko wewnątrz funkcji, nie powoduje zmodyfikowania  
    # globalnej zmiennej x  
    x = 20  
  
funkcja()  
  
print(x)
```

10

```
In [109]: x = 10  
  
def funkcja():  
    # użycie zmiennej globalnej x wewnątrz funkcji bez modyfikacji jej  
    # wartości  
    print(x)  
  
funkcja()  
  
print(x)
```

10

10

Klasy

```
In [110]: class Robot:  
    def __init__(self, name, x, y):  
        # inicjalizacja klasy, wykonuje się zawsze, gdy tworzony jest nowy obiekt  
        # self. - wskazuje na zmienne należące do danej instancji klasy  
        self.name = name  
        self.x = x  
        self.y = y
```

```

def print_pose_info(self): # jedna z metod "print_pose_info" klasy "Robot"
    print("Pose x={:f}, y={:f}".format(self.x, self.y))

def move_right(self):
    self.x += 1

def move_left(self):
    self.x -= 1

def move_up(self):
    self.y += 1

def move_down(self):
    self.y -= 1

def get_distance_to_start(self):
    # metoda zwraca odległość robota od startu (0,0)
    return math.sqrt(self.x*self.x + self.y*self.y)

```

```

In [111]: # Inicjalizacja klas Robot
robot1 = Robot("r1", 0, 0)
robot2 = Robot("r2", 5, 2)

```

```

In [112]: # Odczyt wartości atrybutu klasy
robot2.name

```

```

Out[112]: 'r2'

```

```

In [113]: # Użycie metody klasy
robot2.move_right()
robot2.print_pose_info()

Pose x=6.000000, y=2.000000

```

```

In [114]: # Użycie metody zwracającej jakąś wartość, która została zapisana w zmiennej
# dist_to_start
dist_to_start = robot2.get_distance_to_start()
print("Distance to start: ", dist_to_start)

Distance to start:  6.324555320336759

```

```

In [115]: robot1.print_pose_info()
robot1.move_right()
robot1.print_pose_info()

Pose x=0.000000, y=0.000000
Pose x=1.000000, y=0.000000

```

Klasa z atrybutem, który jest inną klasą

```

In [116]: class Wheel:
    def __init__(self, diameter, width):
        self.diameter = diameter
        self.width = width

class Robot:
    def __init__(self, name, x, y):
        # inicjalizacja klasy, wykonuje się zawsze, gdy tworzony jest nowy obiekt
        # self. - wskazuje na zmienne należące do danej instancji klasy
        self.name = name
        self.x = x
        self.y = y
        self.left_wheel = Wheel(2,1)
        self.right_wheel = Wheel(2,1)

    def print_wheels_diameter(self):
        left_wheel = str(self.left_wheel.diameter)
        right_wheel = str(self.right_wheel.diameter)
        print("Left wheel: " + left_wheel + " right wheel: " + right_wheel)

```

```

In [117]: robot1 = Robot("r1", 0, 0)
# Odczyt średnicy koła lewego
print(robot1.left_wheel.diameter)
# Odczyt średnicy koła prawego
print(robot1.right_wheel.diameter)

```

```

2
2

```

```
In [118]: # utworzenie obiektu koła i przypisanie go do atrybutów z left i right wheel
new_wheel = Wheel(3,4)
robot1 = Robot("r1", 0, 0)
robot1.left_wheel = new_wheel
robot1.right_wheel = new_wheel
robot1.print_wheels_diameter()
```

Left wheel: 3 right wheel: 3

```
In [119]: # Modyfikacja średnicy obiektu koła powoduje modyfikacje kół w robocie
new_wheel.diameter = 2
robot1.print_wheels_diameter()
```

Left wheel: 2 right wheel: 2

```
In [120]: # Modyfikacja średnicy obiektu koła w atrybucie robota również powoduje zmianę
robot1.left_wheel.diameter = 2.4
robot1.print_wheels_diameter()
print("Średnica w obiekcie koła: " + str(new_wheel.diameter))
```

Left wheel: 2.4 right wheel: 2.4
średnica w obiekcie koła: 2.4

Nadpisanie istniejącej metody klasy

```
In [121]: class Wheel:
    def __init__(self, diameter, width):
        self.diameter = diameter
        self.width = width

new_wheel = Wheel(3,4)
print(new_wheel)
```

<__main__.Wheel object at 0x7f4f886e6490>

```
In [122]: class Wheel:
    def __init__(self, diameter, width):
        self.diameter = diameter
        self.width = width

    def __str__(self):
        # Metoda do konwersji klasy na str
        return "Średnica: " + str(self.diameter) + " szerokość: " + str(self.width)

new_wheel = Wheel(3,4)
print(new_wheel)
```

Średnica: 3 szerokość: 4

Nowa nieuzupełniona klasa

```
In [124]: # Błąd przy uruchomieniu kodu
class NowaKlasa:

    File "/tmp/ipykernel_6962/4442539.py", line 2
        class NowaKlasa:
            ^
SyntaxError: unexpected EOF while parsing
```

```
In [125]: # Brak błędu. Możliwość pozostawienia miejsca do późniejszego uzupełnienia w kodzie
# bez konieczności komentowania
class NowaKlasa:
    pass
```

Klasa z Publisherem i Subscriberem - ROS - przykład

```
In [ ]: import rospy
from geometry_msgs.msg import Twist, Pose
rospy.init_node('test')
# Dla danej klasy w projekcie najlepiej będzie utworzyć nowego node'a
# odpowiedzialnego za obsługę interfejsu czy robota.
```

```
In [ ]: class Robot:
    def __init__(self, name, x, y):
```

```

self.name = name
# Dodanie atrybutów z
self.velocity_pub = rospy.Publisher('/cmd_vel', Twist, queue_size=10)
self.pose_sub = rospy.Subscriber('/pose_topic', Pose, self.callback_pose)
self.pose = Pose()

def callback_pose(self, data):
    """
    Metoda wywołuje się gdy na topicu /pose_topic pojawi się dowolna wiadomość.
    Przerwany zostaje aktualnie wykonywany program i aktualizowana jest wartość
    atrybutu pose. W pętli sterującej wartość ta może być wykorzystywana do
    sterowania robotem.
    """
    self.pose = data

def run(self):
    while True:
        msg = Twist()
        self.velocity_pub.publish(msg)

```

```

In [ ]: # wysłanie raz prędkości sterującej
robot1 = Robot("r1", 0, 0)
robot1.run()

```

```

In [ ]: # wysyłanie prędkości sterujących w pętli dopóki roscore jest aktywny
robot1 = Robot("r1", 0, 0)
r = rospy.Rate(10) # 10hz

while not rospy.is_shutdown():
    # pętla odpowiedzialna za wysyłanie prędkości sterujących robotem. Działa
    # z częstotliwością 10Hz
    robot1.run()
    r.sleep()

```

Klasa z Klientem i Serwerem - ROS - przykład

```

In [ ]: from pkg_tsr.srv import Task, TaskResponse
from turtlesim.srv import Spawn

class Robot:
    def __init__(self, name, x, y):
        self.name = name
        # utworzenie serwera /go_to_goal
        self.follow_service = rospy.Service('/go_to_goal', Task, self.go_to_goal)
        self.create_robot(x,y)

    def create_robot(self, x, y):
        # Użycie istniejącego klienta do utworzenia robota w symulacji
        create_new_turtle = rospy.ServiceProxy('spawn', Spawn)
        create_new_turtle(x, y, 0, self.name)

    def go_to_goal(self, msg_srv):
        """
        Tutaj powinien znaleźć się program obsługujący działanie serwera /go_to_goal.
        Po wysłaniu zapytania przez dowolnego klienta na ten serwer powoduje on, że
        robot realizuje zadanie dojazdu do punktu. W zależności od tego czy udało się
        zrealizować dojazd czy nie zwracany jest odpowiedni wynik. Dla kilku robotów
        nazwy serwerów powinny być związane z nazwami różnych robotów.
        """
        # Utworzenie i uzupełnienie odpowiedzi odpowiednimi wartościami.
        response = TaskResponse()

        # zwrócenie wyniku
        return response

```

Klasa z GUI

```

In [127]: import ipywidgets as widgets

class GUI:
    def __init__(self):
        # Atrybuty z kontrolkami: 2 kontrolki do wprowadzania tekstu i kontrolka
        # z przyciskiem
        self.text_a = widgets.Text(value='test', disabled=False,
                                   description="Text A:")
        self.text_b = widgets.Text(value='wersja A', disabled=False,
                                   description="Text B:")
        self.print_button = widgets.Button(
            value=False,
            description='print text',

```

```

        disabled=False,
        button_style='', # 'success', 'info', 'warning', 'danger'
        # or ''
        tooltip='Description',
        icon='check' # (FontAwesome names without the `fa-` prefix)
    )
    # Dodanie obsługi kliknięcia przycisku. Po kliknięciu wywoła się
    # metoda print_text z tej klasy (GUI)
    self.print_button.on_click(self.print_text)

    def display_gui(self):
        # metoda odpowiedzialna za wyświetlenie interfejsu użytkownika
        # na tym poziomie można ustawić opis z tworzeniem siatki z przyciskami
        display(self.text_a)
        display(self.text_b)
        display(self.print_button)

    def print_text(self, button_data):
        # metoda wywołująca się po kliknięciu przycisku z napisem print_text
        print(self.text_a.value + self.text_b.value)

```

```

In [128]: # Utworzenie obiektu GUI
gui = GUI()
# Wyświetlenie interfejsu
gui.display_gui()

```

Text A:

Text B:

✓ print text

In []: